STANDARD OPERATING PROCEDURE

Insider Threat Detection System Local Operations & Run Manual

Comprehensive Guide for Installation, Model Execution, Service Orchestration, and System Verification

Project:	Insider Threat Detection Prototype (Capstone Project A)
Stack:	React (Vite) + FastAPI (Python 3.10+) + MongoDB 7 + Scikit-Learn / XGBoost / SHAP
Scope:	Local Deployment, Machine Learning Pipeline, REST API, Analyst Portal
Target OS:	macOS (Zsh) / Linux (Bash) / Windows (PowerShell)
Default URLs:	Frontend: http://localhost:5173 | API: http://127.0.0.1:8000 (Docs: /docs)

TABLE OF CONTENTS

- 1. System Architecture Overview
- 2. Host Prerequisites & Pre-flight
- 3. Step-by-Step Local Deployment
- 4. Security & Account Provisioning
- 5. Running the Application Services
- 6. Analyst Workflow Walkthrough
- 7. Verification & Diagnostic Suite
- 8. Troubleshooting & Port Reference
- 9. Master Quick-Reference Matrix

SECTION 1

1. System Architecture & Component Model

The Insider Threat Detection System processes user work sessions (0 to 100 risk score) from CERT-layout activity logs. It identifies anomalous behavioral patterns, computes SHAP-based feature contributions, records baseline deviations, and surfaces actionable alerts to security analysts.

Tier	Technology	Network / Port	Responsibility
Frontend Dashboard	React 18, Vite 5	`http://localhost:5173`	Analyst UI: login, flagged sessions triage, session details, user timelines, explainable AI chat.
Backend REST API	FastAPI, Uvicorn, PyJWT	`http://127.0.0.1:8000`	JWT authentication, audit logging, MongoDB queries, AI assistant explanation endpoints.
Database Tier	MongoDB 7.0	`127.0.0.1:27017`	Persists sessions, analyst accounts, audit logs, models (GridFS) with SHA-256 tamper checks.
Detection Engine	XGBoost, Sklearn, SHAP	Batch / CLI	Ingestion, sessionization, baseline deviation extraction, model ensemble training & scoring.

SECTION 2

2. Host Prerequisites & Pre-Flight Verification

Before launching the system, confirm that the required runtime runtimes are installed on your host:

- **Docker & Docker Compose**: Required for containerized MongoDB. Docker Desktop must be running.
- **Python 3.10+**: Required for the backend and ML engine (tested with Python 3.10 – 3.14).
- **Node.js 18+ and npm**: Required for the frontend development server.

> **Pre-Flight Environment Check**
> The repository includes an automated environment validation script. Once you have activated the virtual environment in Section 3, run: `python scripts/check_setup.py`. Every line must return **[OK]**.

SECTION 3

3. Step-by-Step Local Deployment Procedure

Step 1: Start MongoDB Database

From the root of the repository, start the containerized database instance in detached mode:

```
docker compose up -d
```

Verify that the MongoDB container is running and healthy:

```
docker ps --filter "name=mongo"
```

Step 2: Initialize Backend Virtual Environment & Dependencies

Navigate into the `backend/` directory, create an isolated virtual environment, activate it, and install all required packages:

On macOS / Linux (bash/zsh):

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

On Windows PowerShell:

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Execute the automated pre-flight setup check:

```
python scripts/check_setup.py
```

Step 3: Generate Synthetic CERT Activity Logs

Generate synthetic multi-user telemetry (logon, file transfers, HTTP activity, emails, removable device events):

```
python scripts/generate_synthetic_cert.py data/cert_synthetic
```

This creates the test logs in `backend/data/cert_synthetic/` (e.g., `logon.csv`, `file.csv`, `device.csv`, `http.csv`, `email.csv`, `answers.csv`).

Step 4: Execute the Machine Learning Detection Engine

Process the raw telemetry, sessionize actions, calculate baselines, train the ensemble models (XGBoost, Logistic Regression, Isolation Forest), compute SHAP explanations, and persist scored sessions into MongoDB:

```
python -m itd.engine --data data/cert_synthetic --retrain
```

> **Engine Execution Summary**
>
> The command outputs evaluation metrics (Precision, Recall, F1, PR-AUC), saves models with SHA-256 integrity hashes to MongoDB GridFS, and records session anomaly flags.

SECTION 4

4. Security & Account Provisioning

Step 1: Create an Analyst User Account

Analyst credentials are stored in MongoDB with secure bcrypt password hashing:

```
python -m scripts.create_analyst alice "ChooseAStrongOne1!" analyst
```

Step 2: Set the JWT Signing Secret

Export an unpredictable 64-character hexadecimal key in your shell before starting the API:

On macOS / Linux:

```
export JWT_SECRET=$(python3 -c "import secrets; print(secrets.token_hex(32))")
```

On Windows PowerShell:

```
$env:JWT_SECRET = python -c "import secrets; print(secrets.token_hex(32))"
```

SECTION 5

5. Running the Application Services

Running the complete solution requires two active terminal sessions:

Terminal 1: Backend FastAPI Server

```
cd backend
source .venv/bin/activate                    # (On Windows: .\.venv\Scripts\Activate.ps1)
python -m itd.serve
```

- **API Service URL**: `http://127.0.0.1:8000`
- **Interactive OpenAPI Documentation**: `http://127.0.0.1:8000/docs`
- **Health Check Endpoint**: `http://127.0.0.1:8000/api/health`

Terminal 2: Frontend React Dashboard

```
cd frontend
npm install                                  # First time only
npm run dev
```

- **Dashboard URL**: `http://localhost:5173`
- **API Proxy**: The Vite server automatically forwards `/api/*` requests to `http://127.0.0.1:8000`.

SECTION 6

6. Analyst Operational Workflow Walkthrough

Workflow 1: Triage & Flagged Session Investigation

1. Navigate to `http://localhost:5173` and log in as `alice` with password `ChooseAStrongOne1!`.
2. Review the **Flagged Sessions** view, which ranks suspicious work sessions by risk score.
3. Click the highest-scoring session to open the **Session Detail** view. Examine:
    - **Score Ring**: Ensemble risk rating from 0 to 100 against threshold.
    - **What Drove the Score (S1–S3)**: SHAP contributions highlighting anomaly drivers (e.g. zip compression, after-hours logon).
    - **Baseline Deviations (B1–B3)**: Standard deviations away from the user's historical normal.
    - **AI Security Assistant**: Ask questions such as *"Why was this session flagged?"* or *"Were zip files involved?"*.

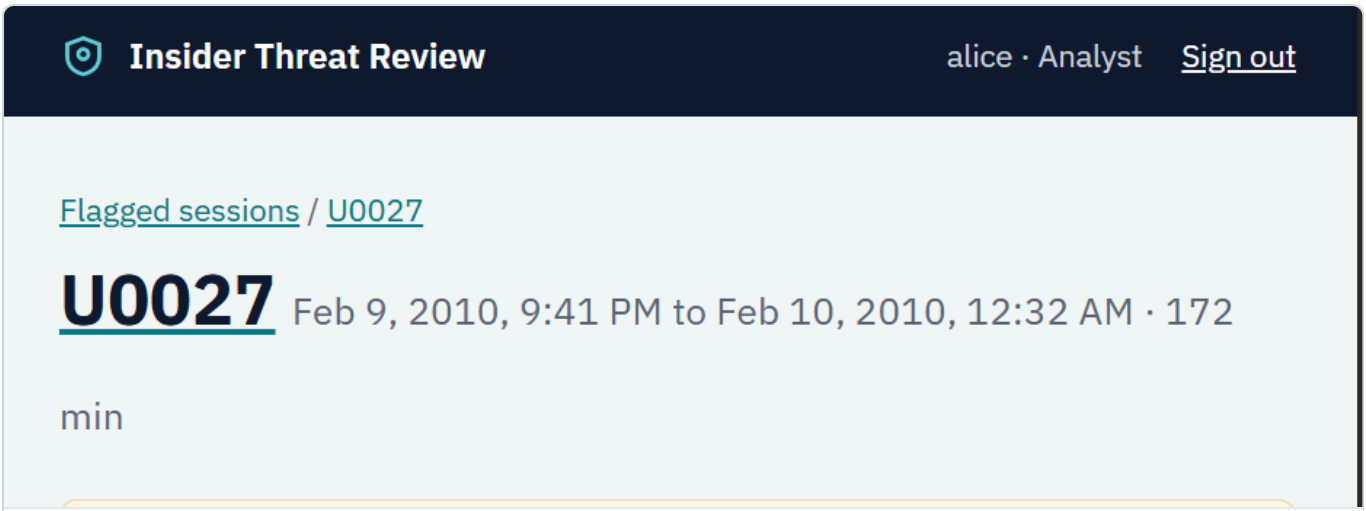


Figure 6.1: Session Detail Interface showing Risk Score Ring, SHAP Drivers (S1–S3), Baseline Deviations (B1–B3), and AI Assistant.

Workflow 2: Longitudinal User Profiling

1. Click the user link (e.g., `U0027`) in the session title.
2. Examine the **Risk Over Time** timeline: grey dots indicate benign work sessions, while large colored markers highlight flagged anomalies.
3. Review the **What Is Normal for This User** table (10 baseline behavioral metrics) and cross-reference previous incidents.

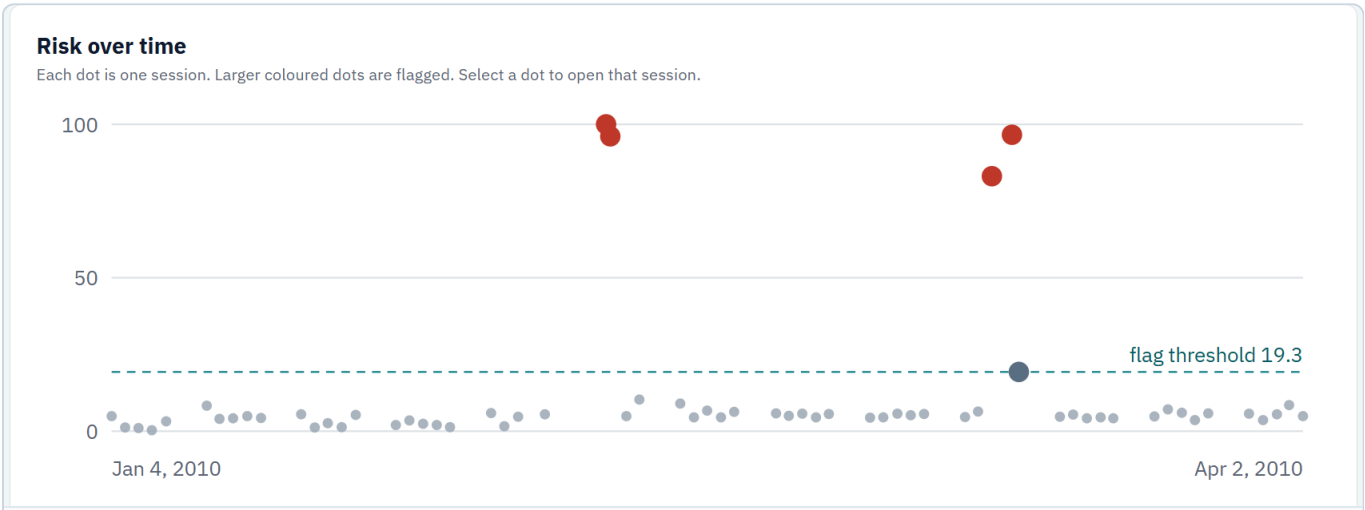


Figure 6.2: Longitudinal Risk-over-Time Timeline for User U0027.

SECTION 7

7. Verification & Diagnostic Testing Suite

The repository provides diagnostic scripts to validate each layer of the application.

1. End-to-End Connection Check

Verifies the database connection, presence of trained models, analyst credentials, API health, and dashboard proxy:

```
cd backend
python scripts/check_connection.py --user alice --password "ChooseAStrongOne1!"
```

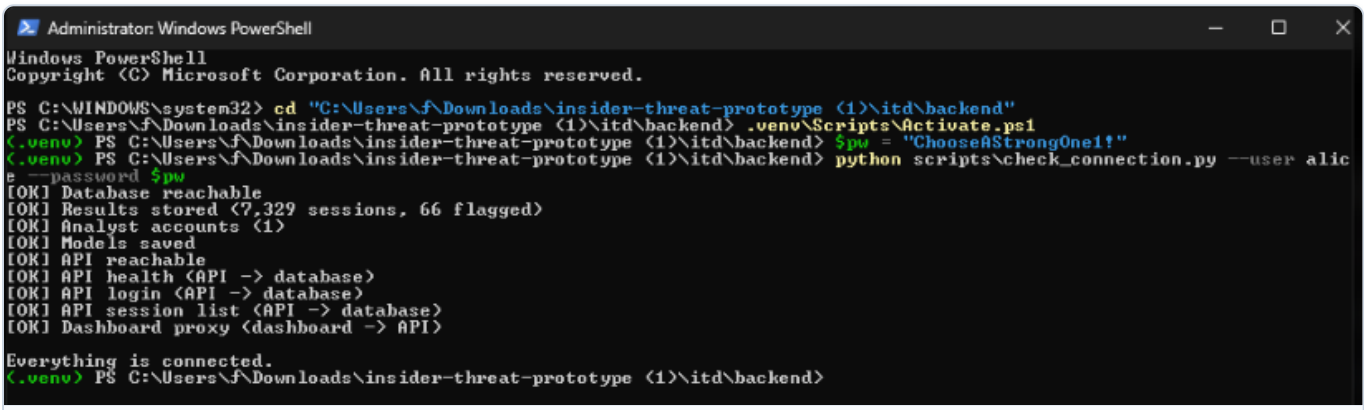


Figure 7.1: Successful execution of check_connection.py verifying all layers.

2. Comprehensive API Diagnostic Run

Calls every REST API endpoint, testing authentication, rate limits, input sanitization, and data retrieval:

```
python scripts/demo_api.py --user alice --password "ChooseAStrongOne1!"
```

3. Automated Unit & Integration Tests

Runs the automated pytest suite using in-memory mock databases (no external MongoDB required):

```
pytest -q
```

4. Database Schema Validation & CRUD Demonstration

```
python -m itd.schema check
python scripts/demo_schema.py
python scripts/demo_db.py
```

SECTION 8

8. Troubleshooting & Common Operational Errors

Symptom / Error	Probable Cause	Resolution Procedure
`[FAIL] Database reachable (ConnectError)`	MongoDB container is stopped or Docker is not running.	Verify Docker Desktop is active. Run `docker compose up -d` from project root.
`HTTP 401: Wrong username or password`	Analyst user not created or password typo.	Run `python -m scripts.create_analyst alice "ChooseAStrongOne1!" analyst`.
`ModelIntegrityError: Saved model failed integrity check`	Model binary modified or SHA-256 hash mismatch.	Retrain and re-sign models with: `python -m itd.engine --data data/cert_synthetic --retrain`.
`Dashboard proxy error (HTTP 500 / ECONNREFUSED)`	Frontend running, but backend API server is stopped.	Start the backend API in Terminal 1 with: `python -m itd.serve`.

SECTION 9

9. Master Quick-Reference Command Matrix

Operational Task	macOS / Linux (Zsh / Bash)	Windows (PowerShell)
Start Database	`docker compose up -d`	`docker compose up -d`
Activate Python Venv	`source backend/.venv/bin/activate`	`.ackend\.venv\Scripts\Activate.ps1`
Install Requirements	`cd backend && pip install -r requirements.txt`	`cd backend; pip install -r requirements.txt`
Generate Synthetic Data	`python scripts/generate_synthetic_cert.py data/cert_synthetic`	`python scripts\generate_synthetic_cert.py data\cert_synthetic`
Train Models & Score	`python -m itd.engine --data data/cert_synthetic --retrain`	`python -m itd.engine --data data\cert_synthetic --retrain`
Create Analyst User	`python -m scripts.create_analyst alice "ChooseAStrongOne1!" analyst`	`python -m scripts.create_analyst alice "ChooseAStrongOne1!" analyst`
Set JWT Secret	`export JWT_SECRET=$(python3 -c "import secrets; print(secrets.token_hex(32))")`	`$env:JWT_SECRET = python -c "import secrets; print(secrets.token_hex(32))"`
Run Backend API	`cd backend && python -m itd.serve`	`cd backend; python -m itd.serve`
Run Frontend Dashboard	`cd frontend && npm run dev`	`cd frontend; npm run dev`
Check Connections	`python scripts/check_connection.py --user alice --password "ChooseAStrongOne1!"`	`python scripts\check_connection.py --user alice --password "ChooseAStrongOne1!"`
Run Automated Tests	`pytest -q`	`pytest -q`
Stop All Services	`docker compose down` (Ctrl+C in API / Web terminals)	`docker compose down` (Ctrl+C in API / Web terminals)